Guide du Code et de la Demonstration

Application Web — XYZ University Student Records

Salim LAKHAL & Nossa IYAMU

CSC 8604 — Cloud Computing | Telecom SudParis

Année universitaire 2025–2026

1. Vue d'ensemble

L'application est une CRUD web Node.js déployée sur des instances EC2 dans un Auto Scaling Group, accessible via un Application Load Balancer (ALB). Les données sont stockées dans une base RDS MySQL, et les credentials sont gérés de façon sécurisée via AWS Secrets Manager.

Paramètre	Valeur
URL d'accès	`http://Lab-ALB-937683819.us-east-1.elb.amazonaws.com/`
Port	80

Paramètre	Valeur
Runtime	Node.js 18 + Express.js 4.18
BDD	Amazon RDS MySQL 8.0
Secrets	AWS Secrets Manager (`Mydbsecret`)
Région	us-east-1

2. Architecture des fichiers

L'arborescence de l'application sur le serveur EC2 :

```
/home/ubuntu/app/

├── package.json

├── app.js

└── views/

    ├── index.ejs

    ├── add.ejs

    └── edit.ejs
```

2.1 package.json

```
{

  "name": "student-app",
```

```
  "version": "1.0.0",

  "dependencies": {

    "express": "^4.18.2",

    "mysql2": "^3.6.0",

    "ejs": "^3.1.9",

    "@aws-sdk/client-secrets-manager": "^3.0.0"

  }

}
```

Description des dépendances :

- **express** — Framework HTTP minimaliste pour Node.js, gère le routage et les middlewares.
- **mysql2** — Pilote MySQL haute performance avec support des requêtes préparées et des Promises.
- **ejs** — Moteur de templates permettant de générer du HTML dynamique côté serveur.
- **@aws-sdk/client-secrets-manager** — SDK AWS officiel pour récupérer les secrets depuis AWS Secrets Manager.

3. Code source commenté (app.js)

3.1 Imports et initialisation

```
const express = require('express');

const mysql = require('mysql2');
```

```
const { SecretsManagerClient, GetSecretValueCommand } = require("@aws-sdk/client-secrets-manager");

const app = express();

app.set('view engine', 'ejs');

app.use(express.urlencoded({ extended: true }));
```

On importe Express, le driver MySQL2 et le client Secrets Manager. L'application est configurée pour utiliser EJS comme moteur de rendu, et le middleware `urlencoded` permet de parser les données des formulaires HTML transmises en POST.

3.2 Récupération des credentials via Secrets Manager

```
let db;

async function getSecret() {

  const client = new SecretsManagerClient({ region: "us-east-1" });

  const response = await client.send(

    new GetSecretValueCommand({ SecretId: "Mydbsecret" })

  );

  return JSON.parse(response.SecretString);

}
```

Cette fonction contacte AWS Secrets Manager pour récupérer le secret nommé `Mydbsecret`. Ce secret contient en JSON les champs `host`, `user`, `password`, `database` et `port` nécessaires à la connexion RDS. Pour que l'appel réussisse, l'instance EC2 doit avoir le rôle IAM **LabRole** attaché, qui confère les permissions `secretsmanager:GetSecretValue`.

3.3 Connexion à la base de données

```
async function initDB() {

  const secret = await getSecret();

  db = mysql.createConnection({

    host: secret.host,

    user: secret.user,

    password: secret.password,

    database: secret.database,

    port: secret.port || 3306

  });

  db.connect((err) => {

    if (err) { console.error('DB error:', err); process.exit(1); }

    console.log('Connected to RDS MySQL');

    db.query(`CREATE TABLE IF NOT EXISTS students (

      id INT AUTO_INCREMENT PRIMARY KEY,

      name VARCHAR(255), address VARCHAR(255),

      city VARCHAR(255), state VARCHAR(255),

      email VARCHAR(255), phone VARCHAR(20)

    )`);
```

```
  });

}
```

`initDB()` orchestre le démarrage : elle appelle `getSecret()` pour obtenir les credentials, crée la connexion MySQL, puis exécute un `CREATE TABLE IF NOT EXISTS` afin de s'assurer que la table `students` existe dès le premier lancement. En cas d'échec de connexion, le processus se termine avec code 1.

3.4 Routes CRUD

L'application expose six routes HTTP qui couvrent les quatre opérations CRUD. Les requêtes SQL utilisent des **requêtes préparées** (paramètre `?`) afin de protéger contre les injections SQL.

GET / — Lister tous les étudiants

```
app.get('/', (req, res) => {

  db.query('SELECT * FROM students', (err, results) => {

    if (err) return res.status(500).send('Database error');

    res.render('index', { students: results });

  });

});
```

Récupère tous les enregistrements et les transmet au template `index.ejs`.

GET /add — Afficher le formulaire d'ajout

```
app.get('/add', (req, res) => {
```

```
  res.render('add');

});
```

Affiche simplement le formulaire vide `add.ejs`.

POST /add — Insérer un étudiant

```
app.post('/add', (req, res) => {

  const { name, address, city, state, email, phone } = req.body;

  db.query(

    'INSERT INTO students (name, address, city, state, email, phone) VALUES (?, ?, ?,
?, ?, ?)',

    [name, address, city, state, email, phone],

    (err) => {

      if (err) return res.status(500).send('Insert error');

      res.redirect('/');

    }

  );

});
```

Extrait les champs du body du formulaire et les insère via une requête préparée, puis redirige vers l'accueil.

GET /edit/:id — Afficher le formulaire de modification

```
app.get('/edit/:id', (req, res) => {

  db.query('SELECT * FROM students WHERE id = ?', [req.params.id], (err, results) =>
{

    if (err || results.length === 0) return res.status(404).send('Student not
found');

    res.render('edit', { student: results[0] });

  });

});
```

Charge l'étudiant par son identifiant et pré-remplit le formulaire `edit.ejs`.

POST /edit/:id — Mettre à jour un étudiant

```
app.post('/edit/:id', (req, res) => {

  const { name, address, city, state, email, phone } = req.body;

  db.query(

    'UPDATE students SET name=?, address=?, city=?, state=?, email=?, phone=? WHERE
id=?',

    [name, address, city, state, email, phone, req.params.id],

    (err) => {

      if (err) return res.status(500).send('Update error');

      res.redirect('/');

    }

  );
```

```
});
```

Met à jour tous les champs de l'étudiant identifié par l'URL, puis redirige.

GET /delete/:id — Supprimer un étudiant

```
app.get('/delete/:id', (req, res) => {

  db.query('DELETE FROM students WHERE id = ?', [req.params.id], (err) => {

    if (err) return res.status(500).send('Delete error');

    res.redirect('/');

  });

});
```

Supprime l'étudiant correspondant à l'identifiant fourni dans l'URL et redirige vers l'accueil.

3.5 Démarrage du serveur

```
initDB().then(() => {

  app.listen(80, () => { console.log('Server running on port 80'); });

}).catch(err => { console.error('Init failed:', err); process.exit(1); });
```

Le serveur ne démarre l'écoute sur le port 80 qu'après que la base de données a été initialisée avec succès. Si `initDB()` échoue (réseau, credentials incorrects, etc.), le processus se termine immédiatement avec un code d'erreur.

4. Vues EJS

4.1 index.ejs — Page d'accueil

La vue `index.ejs` reçoit le tableau `students` injecté par la route `GET /`. Elle génère un tableau HTML listant tous les étudiants avec leurs champs (name, address, city, state, email, phone). Chaque ligne dispose de deux liens : **Edit** (renvoyant vers `/edit/:id`) et **Delete** (renvoyant vers `/delete/:id`). Un lien *« Add a new student »* en haut de page pointe vers `/add`.

4.2 add.ejs — Formulaire d'ajout

Formulaire HTML simple avec six champs de saisie : `name`, `address`, `city`, `state`, `email` et `phone`. L'attribut `action` du formulaire est `/add` avec la méthode `POST`. Un bouton *Submit* valide la saisie.

4.3 edit.ejs — Formulaire de modification

Identique à `add.ejs` en structure, mais les champs sont pré-remplis avec les valeurs de l'étudiant récupéré (`student.name`, `student.address`, etc.) grâce aux balises EJS `<%= student.field %>`. L'action du formulaire pointe vers `/edit/<%= student.id %>` en méthode `POST`. Un bouton *Update* soumet les modifications.

5. Service systemd

L'application est gérée par systemd, ce qui garantit son démarrage automatique au boot de l'instance et son redémarrage en cas de crash. Le fichier de service est installé dans `/etc/systemd/system/student-app.service`.

```
[Unit]

Description=Student App
```

```
After=network.target




[Service]

Type=simple

User=root

WorkingDirectory=/home/ubuntu/app

ExecStart=/usr/local/bin/node app.js

Restart=on-failure

RestartSec=5




[Install]

WantedBy=multi-user.target
```

Points clés :

- **After=network.target** — S'assure que la couche réseau est disponible avant de démarrer l'application (nécessaire pour atteindre Secrets Manager et RDS).
- **Restart=on-failure** — Redémarre automatiquement le processus s'il se termine avec un code non nul.
- **RestartSec=5** — Attend 5 secondes avant de tenter un redémarrage, évitant une boucle de crash trop rapide.
- **WantedBy=multi-user.target** — Active le service au démarrage en mode multi-utilisateur standard.

> *Commandes de gestion : `systemctl enable student-app` pour activer au boot, `systemctl`*

> *`start student-app` pour démarrer, `journalctl -u student-app -f` pour suivre les logs.*

6. Guide de démonstration pas à pas

> ***Prérequis :*** *Le lab doit être actif dans AWS Academy avant toute démonstration. Les ressources AWS (EC2, RDS, ALB) ne sont disponibles que lorsque le lab est en état Started.*

> **Étape 1 — Vérifier que le lab est actif** Se connecter à Canvas → AWS Academy → ouvrir le lab → cliquer *Start Lab* → attendre que l'indicateur passe au vert. Ne pas fermer la session pendant la démo.

> **Étape 2 — Accéder via l'URL de l'ALB** Ouvrir un navigateur et accéder à `http://Lab-ALB-937683819.us-east-1.elb.amazonaws.com/`. La page d'accueil listant les étudiants doit s'afficher, confirmant que l'ALB, les EC2 et RDS fonctionnent.

> **Étape 3 — Démontrer Read (lecture)** La page d'accueil liste automatiquement tous les étudiants présents dans la base. Si la table est vide, c'est normal — les étapes suivantes vont la peupler.

> **Étape 4 — Démontrer Create (création)** Cliquer sur *« Add a new student »* → remplir les 6 champs du formulaire (nom, adresse, ville, état, email, téléphone) → cliquer *Submit*. La liste se raffraîchit avec le nouvel enregistrement.

> **Étape 5 — Démontrer Update (mise à jour)** Sur la ligne d'un étudiant, cliquer le lien *Edit* → modifier un ou plusieurs champs → cliquer *Update*. Vérifier que la modification est bien reflétée dans la liste.

> **Étape 6 — Démontrer Delete (suppression)** Cliquer le lien *Delete* sur n'importe quel étudiant. L'enregistrement disparaît immédiatement de la liste.

> **Étape 7 — Démontrer la haute disponibilité** Dans la console AWS → EC2 → Instances, montrer les deux instances en cours d'exécution dans deux zones de disponibilité

> différentes (us-east-1a et us-east-1b). Pour aller plus loin, terminer une instance : l'ASG en redémarre automatiquement une nouvelle, et l'ALB continue de router le trafic vers l'instance restante sans interruption de service.

> **Étape 8 — Montrer l'architecture dans la console AWS** Parcourir les services suivants pour illustrer l'architecture :

- **VPC** — réseaux public/privé, subnets, IGW
- **EC2 → Auto Scaling Groups** — politique de scaling, launch template
- **EC2 → Load Balancers** — listeners, target group, health checks
- **RDS** — instance MySQL, subnet group, SG DB
- **Secrets Manager** — secret `Mydbsecret`, rotation possible

> **Étape 9 — Démontrer le scaling (optionnel)** Depuis une instance EC2, exécuter un test de charge pour déclencher le scaling :

```
ab -n 50000 -c 200 http://Lab-ALB-937683819.us-east-1.elb.amazonaws.com/
```

> Observer dans la console EC2 l'augmentation du nombre d'instances dans l'ASG lorsque le seuil CPU est dépassé.

7. Configuration du secret

Le secret est créé dans AWS Secrets Manager avec la commande AWS CLI suivante :

```
aws secretsmanager create-secret \

  --name Mydbsecret \
```

```
  --region us-east-1 \

  --secret-string '{

    "host":     "lab-db.cpe8i6ikwz9f.us-east-1.rds.amazonaws.com",

    "user":     "admin",

    "password": "lab-password",

    "database": "STUDENTS",

    "port":     3306

  }'
```

Ce secret peut ensuite être mis à jour avec `aws secretsmanager put-secret-value` sans redéployer l'application (redémarrer simplement le service systemd pour recharger les credentials).

8. Schéma de la base de données

8.1 Instruction CREATE TABLE

```
CREATE TABLE IF NOT EXISTS students (

  id      INT AUTO_INCREMENT PRIMARY KEY,

  name    VARCHAR(255),

  address VARCHAR(255),

  city    VARCHAR(255),

  state   VARCHAR(255),
```

```
  email   VARCHAR(255),

  phone   VARCHAR(20)

);
```

8.2 Description des colonnes

Colonne	Type	Contrainte	Description
`id`	INT	AUTO_INCREMENT, PRIMARY KEY	Identifiant unique auto-incrémenté
`name`	VARCHAR(255)	—	Nom complet de l'étudiant
`address`	VARCHAR(255)	—	Adresse postale
`city`	VARCHAR(255)	—	Ville de résidence
`state`	VARCHAR(255)	—	État ou province
`email`	VARCHAR(255)	—	Adresse e-mail
`phone`	VARCHAR(20)	—	Numéro de téléphone

9. Identifiants des ressources AWS

Ces identifiants correspondent aux ressources déployées dans le lab AWS Academy pour le compte *Salim LAKHAL & Nossa IYAMU*.

Ressource	Identifiant / Valeur
VPC	`vpc-015f10c548e076bc6`

Ressource	Identifiant / Valeur
IGW	`igw-0b4f40e1ae08c1c8e`
Subnet Public AZ-1	`subnet-0c9a86af8623ce46b`
Subnet Public AZ-2	`subnet-0dd7aa0d705aeddb9`
Subnet Privé AZ-1	`subnet-09fb98871d85efcdc`
Subnet Privé AZ-2	`subnet-0f8e7e531bd61e31f`
SG Web	`sg-063f2429eee0eb830`
SG DB	`sg-0e8e5a2d8a5f7b3d4`
SG ALB	`sg-0522cafcbcdd82dc0`
RDS Endpoint	`lab-db.cpe8i6ikwz9f.us-east-1.rds.amazonaws.com`
ALB DNS	`Lab-ALB-937683819.us-east-1.elb.amazonaws.com`
AMI v2	`ami-00e8094efe00b85cf`
Launch Template	`lt-0db39dbf8cf32b055`
Target Group ARN	`arn:aws:elasticloadbalancing:us-east-1::targetgroup/Lab-TG/90bbc260df64a90b`
ALB ARN	`arn:aws:elasticloadbalancing:us-east-1::loadbalancer/app/Lab-ALB/edd3e33aa527fa25`